Rapport SAE 104 :

Création base de données : Rapportage de la saison balnéaire 2024

MOHAMMAD Muhammad Mousa
Mercure
Promotion 2025 : BUT

1 : Script manuel de création de la base de données

```sql
DROP TABLE IF EXISTS analyses CASCADE;
DROP TABLE IF EXISTS evenements CASCADE;
DROP TABLE IF EXISTS sites CASCADE;
DROP TABLE IF EXISTS communes CASCADE;
DROP TABLE IF EXISTS departements CASCADE;
DROP TABLE IF EXISTS regions CASCADE;
DROP TYPE IF EXISTS type_eau_enum CASCADE;

CREATE TABLE regions (
    code_region VARCHAR(3) PRIMARY KEY,
    nom_region TEXT
);

CREATE TABLE departements (
    code_departement VARCHAR(3) PRIMARY KEY,
    nom_departement TEXT,
    region VARCHAR(3),
    CONSTRAINT fk_regions_departements
    FOREIGN KEY (region) REFERENCES regions(code_region)
);

CREATE TABLE communes (
    code_commune VARCHAR(5) PRIMARY KEY,
    nom_commune TEXT,
    departement VARCHAR(3),
    CONSTRAINT fk_departements_commune
    FOREIGN KEY (departement) REFERENCES departements(code_departement)
);

CREATE TYPE type_eau_enum AS ENUM ('Rivière', 'Eau côtière', 'Lac', 'Eau de
transition');
CREATE TABLE sites (
    idSite TEXT PRIMARY KEY,
    nom_site TEXT,
    commune VARCHAR(5),
    dateDeclaration DATE,
    typeEau type_eau_enum,
    longitude REAL,
    latitude REAL,
    CONSTRAINT fk_communes_sites
    FOREIGN KEY (commune) REFERENCES communes(code_commune)
);

CREATE TABLE evenements (
```

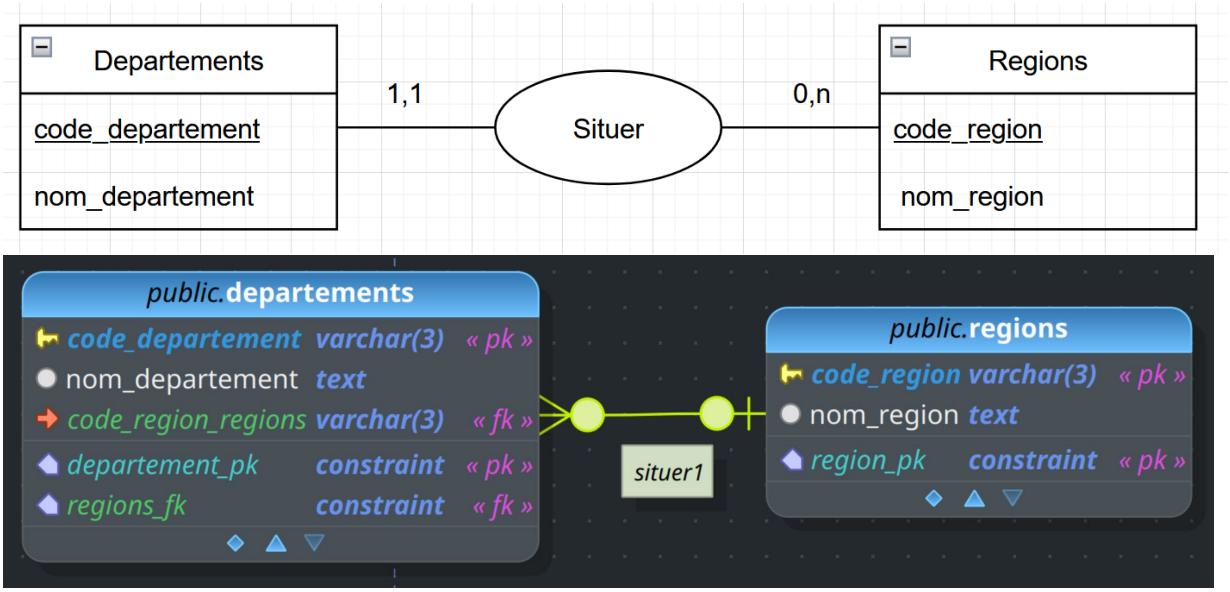
```
    idEvenement SERIAL PRIMARY KEY,
    idSite TEXT,
    evenement TEXT,
    debut DATE,
    fin DATE,
    mesure TEXT,
    CONSTRAINT fk_sites_evenements
    FOREIGN KEY (idSite) REFERENCES sites(idSite)
);

CREATE TABLE analyses (
    idAnalyse SERIAL PRIMARY KEY,
    idSite TEXT,
    datePrelevement DATE,
    enterocoques INTEGER,
    escherichia INTEGER,
    CONSTRAINT fk_sites_analyses
    FOREIGN KEY (idSite) REFERENCES sites(idSite)
);
```

---

2 : Modélisation et script de création avec PgModeler

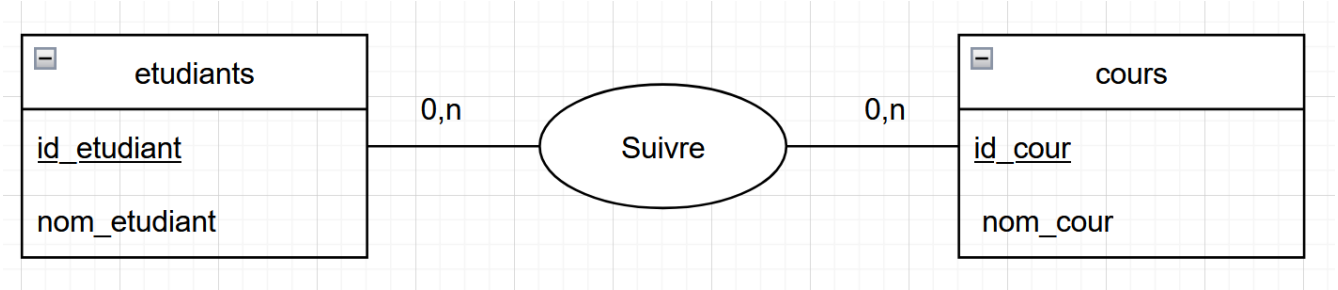
Relation fonctionnelle :

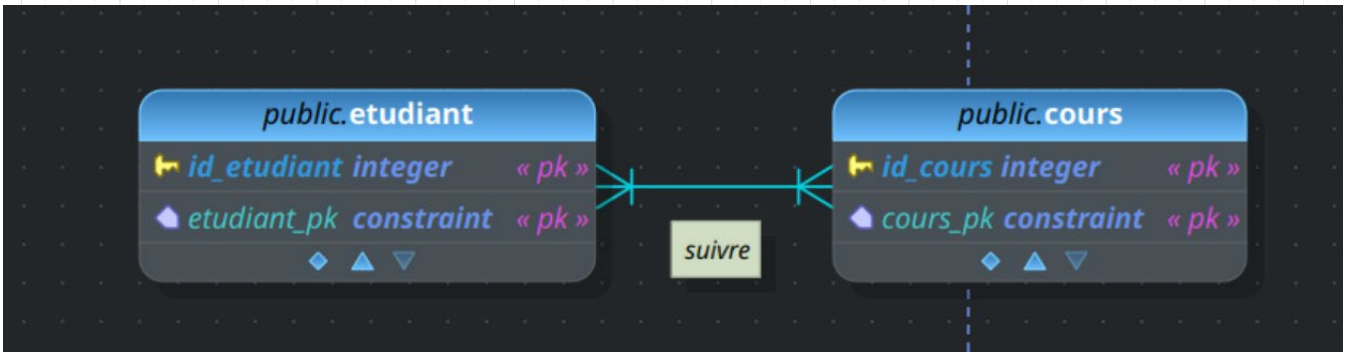


Dans le modèle conceptuel de données, la relation est représentée par un ovale nommé « Situer ». De plus, les clés primaires sont reconnues car elles sont soulignées. Enfin, la cardinalité est explicitée et c'est elle qui nous permet de prendre connaissance des clés étrangères. En effet, la cardinalité 1,1 du côté de « departement » nous indique la présence d'une clé étrangère dans celle-ci.

Dans le modèle physique, la relation est représentée par la ligne entre les deux tables et la « patte d'oie » du côté de « departement » indique l'association fonctionnelle. De plus, les clés primaires sont marquées par le tab « pk » pour « primary key » et les clés étrangères par le tag « fk » pour « foreign key ». Enfin, les types des éléments sont également spécifiés.

Relation maillée :

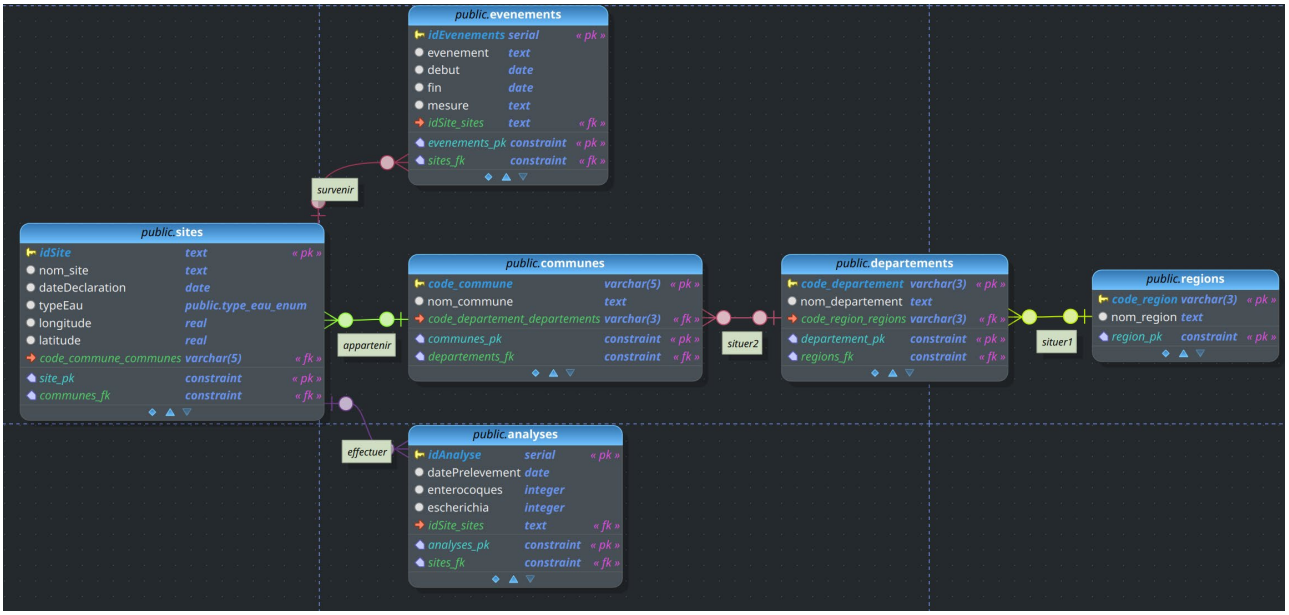




Dans le modèle conceptuel de données, la relation est représentée par un ovale nommé « Suivre ». De plus, les clés primaires sont reconnues car elles sont soulignées. Enfin, la cardinalité est explicitée et c'est elle qui nous permet de prendre connaissance de l'association maillée. En effet, la cardinalité 0,n des deux côtés nous indique la présence de clés étrangères dans les deux tables.

Dans le modèle physique, la relation est représentée par la ligne entre les deux tables et la « patte d'oie » des deux côtés indique l'association maillée. De plus, les clés primaires sont marquées par le tab « pk » pour « primary key » et les clés étrangères par le tag « fk » pour « foreign key ». Enfin, les types des éléments sont également spécifiés.

Modèle physique de données réalisé avec l'AGL :



Script de création des tables généré automatiquement par l'AGL :

```
-- ** Database generated with pgModeler (PostgreSQL Database Modeler).
-- ** pgModeler version: 1.2.2
-- ** PostgreSQL version: 18.0
-- ** Project Site: pgmodeler.io
-- ** Model Author: ---

-- ** Database creation must be performed outside a multi lined SQL file.
-- ** These commands were put in this file only as a convenience.

-- object: "BD_SAE" | type: DATABASE --
-- DROP DATABASE IF EXISTS "BD_SAE";
CREATE DATABASE "BD_SAE";
-- ddl-end --


SET search_path TO pg_catalog,public;
-- ddl-end --

-- object: public.regions | type: TABLE --
-- DROP TABLE IF EXISTS public.regions CASCADE;
CREATE TABLE public.regions (
    code_region varchar(3) NOT NULL,
    nom_region text,
    CONSTRAINT region_pk PRIMARY KEY (code_region)
);
-- ddl-end --
ALTER TABLE public.regions OWNER TO postgres;
-- ddl-end --
```

```sql
-- object: public.departements | type: TABLE --
-- DROP TABLE IF EXISTS public.departements CASCADE;
CREATE TABLE public.departements (
    code_departement varchar(3) NOT NULL,
    nom_departement text,
    code_region_regions varchar(5),
    CONSTRAINT departement_pk PRIMARY KEY (code_departement)
);
-- ddl-end --
ALTER TABLE public.departements OWNER TO postgres;
-- ddl-end --

-- object: regions_fk | type: CONSTRAINT --
-- ALTER TABLE public.departements DROP CONSTRAINT IF EXISTS regions_fk CASCADE;
ALTER TABLE public.departements ADD CONSTRAINT regions_fk FOREIGN KEY
(code_region_regions)
REFERENCES public.regions (code_region) MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --

-- object: public.communes | type: TABLE --
-- DROP TABLE IF EXISTS public.communes CASCADE;
CREATE TABLE public.communes (
    code_commune varchar(5) NOT NULL,
    nom_commune text,
    code_departement_departements varchar(3),
    CONSTRAINT communes_pk PRIMARY KEY (code_commune)
);
-- ddl-end --
ALTER TABLE public.communes OWNER TO postgres;
-- ddl-end --

-- object: departements_fk | type: CONSTRAINT --
-- ALTER TABLE public.communes DROP CONSTRAINT IF EXISTS departements_fk CASCADE;
ALTER TABLE public.communes ADD CONSTRAINT departements_fk FOREIGN KEY
(code_departement_departements)
REFERENCES public.departements (code_departement) MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --

-- object: public.evenements | type: TABLE --
-- DROP TABLE IF EXISTS public.evenements CASCADE;
CREATE TABLE public.evenements (
    "idEvenements" serial NOT NULL,
    evenement text,
    debut date,
    fin date,
    mesure text,
```

```sql
    "idSite_sites" text,
    CONSTRAINT evenements_pk PRIMARY KEY ("idEvenements")
);
-- ddl-end --
ALTER TABLE public.evenements OWNER TO postgres;
-- ddl-end --

-- object: public.analyses | type: TABLE --
-- DROP TABLE IF EXISTS public.analyses CASCADE;
CREATE TABLE public.analyses (
    "idAnalyse" serial NOT NULL,
    "datePrelevement" date,
    enterocoques integer,
    escherichia integer,
    "idSite_sites" text,
    CONSTRAINT analyses_pk PRIMARY KEY ("idAnalyse")
);
-- ddl-end --
ALTER TABLE public.analyses OWNER TO postgres;
-- ddl-end --

-- object: public.type_eau_enum | type: TYPE --
-- DROP TYPE IF EXISTS public.type_eau_enum CASCADE;
CREATE TYPE public.type_eau_enum AS
ENUM ('Rivière','Eau côtière','Lac','Eau de transition');
-- ddl-end --
ALTER TYPE public.type_eau_enum OWNER TO postgres;
-- ddl-end --

-- object: public.sites | type: TABLE --
-- DROP TABLE IF EXISTS public.sites CASCADE;
CREATE TABLE public.sites (
    "idSite" text NOT NULL,
    nom_site text,
    "dateDeclaration" date,
    "typeEau" public.type_eau_enum,
    longitude real,
    latitude real,
    code_commune_communes varchar(5),
    CONSTRAINT site_pk PRIMARY KEY ("idSite")
);
-- ddl-end --
ALTER TABLE public.sites OWNER TO postgres;
-- ddl-end --

-- object: sites_fk | type: CONSTRAINT --
-- ALTER TABLE public.evenements DROP CONSTRAINT IF EXISTS sites_fk CASCADE;
ALTER TABLE public.evenements ADD CONSTRAINT sites_fk FOREIGN KEY ("idSite_sites")
REFERENCES public.sites ("idSite") MATCH FULL
```

```sql
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --

-- object: communes_fk | type: CONSTRAINT --
-- ALTER TABLE public.sites DROP CONSTRAINT IF EXISTS communes_fk CASCADE;
ALTER TABLE public.sites ADD CONSTRAINT communes_fk FOREIGN KEY
(code_commune_communes)
REFERENCES public.communes (code_commune) MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --

-- object: sites_fk | type: CONSTRAINT --
-- ALTER TABLE public.analyses DROP CONSTRAINT IF EXISTS sites_fk CASCADE;
ALTER TABLE public.analyses ADD CONSTRAINT sites_fk FOREIGN KEY ("idSite_sites")
REFERENCES public.sites ("idSite") MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --
```

Différences entre le script manuel et le script généré par l'AGL :

Le script manuel est linéaire et direct, il définit les relations dès la création de la table et facilite donc la lecture. De plus, il possède des noms court et facile à comprendre.
Le script de l'AGL permet quant à lui de faciliter les requêtes ainsi que les modifications de la base de données. Il créer d'abord les tables puis les modifie. Les noms attribués sont également sensibles à la casse.

3 : Peuplement des tables

Script de création des tables :

```sql
DROP TABLE IF EXISTS import_analyses CASCADE;
DROP TABLE IF EXISTS import_evenements CASCADE;
DROP TABLE IF EXISTS import_sites CASCADE;
DROP TABLE IF EXISTS import_regions_depts CASCADE;

CREATE TEMP TABLE import_regions_depts (
    code_departement TEXT,
    nom_departement TEXT,
    code_region TEXT,
    nom_region TEXT
);

CREATE TEMP TABLE import_sites (
    saison TEXT,
    region TEXT,
```

```sql
    departement TEXT,
    id_site TEXT,
    prec_id TEXT,
    evolution TEXT,
    nom_site TEXT,
    code_insee_commune TEXT,
    nom_commune TEXT,
    date_declaration TEXT,
    type_eau TEXT,
    longitude TEXT,
    latitude TEXT
);

CREATE TEMP TABLE import_evenements (
    saison TEXT,
    region TEXT,
    departement TEXT,
    id_site TEXT,
    type_evenement TEXT,
    date_debut TEXT,
    date_fin TEXT,
    mesures_gestion TEXT
);

CREATE TEMP TABLE import_analyses (
    saison TEXT,
    region TEXT,
    departement TEXT,
    id_site TEXT,
    date_prelevement TEXT,
    resultat_entero TEXT,
    resultat_ecoli TEXT,
    statut_prelevement TEXT,
    temp1 TEXT,
    temp2 TEXT,
    temp3 TEXT
);

\copy import_regions_depts FROM 'csv_SAE/departements-france.csv' WITH (FORMAT
csv, HEADER, DELIMITER ',', ENCODING 'UTF8');
\copy import_sites FROM 'csv_SAE/liste-des-sites-de-baignade-saison-balneaire-
2024.csv' WITH (FORMAT csv, HEADER, DELIMITER ';', ENCODING 'LATIN1');
\copy import_evenements FROM 'csv_SAE/saison-balneaire-2024-informations-sur-la-
saison.csv' WITH (FORMAT csv, HEADER, DELIMITER ';', ENCODING 'LATIN1');
\copy import_analyses FROM 'csv_SAE/saison-balneaire-2024-resultats-danalyses.csv'
WITH (FORMAT csv, HEADER, DELIMITER ';', ENCODING 'LATIN1');

-- Régions
INSERT INTO regions (code_region, nom_region)
```

```sql
SELECT DISTINCT code_region, nom_region
FROM import_regions_depts;

-- Départements
INSERT INTO departements (code_departement, nom_departement, region)
SELECT DISTINCT code_departement, nom_departement, code_region
FROM import_regions_depts;

-- Communes
INSERT INTO communes (code_commune, nom_commune, departement)
SELECT DISTINCT
    LPAD(code_insee_commune, 5, '0'),
    nom_commune,
    REGEXP_REPLACE(departement, '^0', '')
FROM import_sites
WHERE code_insee_commune IS NOT NULL;

-- Sites de baignade
INSERT INTO sites (idSite, nom_site, commune, dateDeclaration, typeEau, longitude,
latitude)
SELECT DISTINCT
    id_site,
    nom_site,
    LPAD(code_insee_commune, 5, '0'),
    TO_DATE(date_declaration, 'DD/MM/YYYY'),
    type_eau::type_eau_enum,
    REPLACE(longitude, ',', '.')::REAL,
    REPLACE(latitude, ',', '.')::REAL
FROM import_sites
WHERE code_insee_commune IS NOT NULL;

-- Évènements
INSERT INTO evenements (idSite, evenement, debut, fin, mesure)
SELECT
    id_site,
    type_evenement,
    TO_DATE(date_debut, 'DD/MM/YYYY'),
    TO_DATE(date_fin, 'DD/MM/YYYY'),
    mesures_gestion
FROM import_evenements;

-- Analyses
INSERT INTO analyses (idSite, datePrelevement, enterocoques, escherichia)
SELECT
    id_site,
    TO_DATE(date_prelevement, 'DD/MM/YYYY'),
    resultat_entero::INTEGER,
    resultat_ecoli::INTEGER
FROM import_analyses;
```

```sql
DROP TABLE import_analyses;
DROP TABLE import_evenements;
DROP TABLE import_sites;
DROP TABLE import_regions_depts;
```

Description du script :

Nous allons dans une première partie supprimer les tables temporaires si elles existent déjà.
Une fois que nous sommes sûr qu'elles n'existent pas, nous allons les créer de façon à pouvoir accueillir l'entièreté des données contenues dans nos fichiers csv.

Ensuite, nous allons remplir les tables temporaires avec la méthode copy. Cela nous permettra ensuite de n'insérer que les colonnes des tables temporaires qui nous intéressent dans nos tables permanentes.
Durant l'insertion des données dans nos tables, nous allons également rendre ces données conformes au format que nous souhaitons avoir.